

PRODUCTION MARKETPLACE PLATFORM

Master Development Prompt

Full-stack Flutter + Node.js + TypeScript + Express + MySQL marketplace specification

PRODUCTION MARKETPLACE PLATFORM — MASTER DEVELOPMENT PROMPT

0. ROLE AND OBJECTIVE

Act as a senior full-stack software architect and production engineer.

Build a complete, production-ready, fully connected multi-vendor marketplace application.

This is NOT a UI prototype, mockup, demo, static frontend, or collection of disconnected screens.

The final system must be a functioning application where:

Flutter Mobile App

↓

REST API

↓

Node.js + TypeScript + Express

↓

Service Layer

↓

Repository/Data Access Layer

↓

MySQL

Every important frontend workflow must communicate with the backend through real APIs and persist data in MySQL.

Do not use fake data, hardcoded marketplace data, fake authentication, simulated checkout, local-only orders, or placeholder business logic except where explicitly identified as a temporary integration adapter.

1. NON-NEGOTIABLE REQUIREMENTS

You MUST:

1. Build the backend and frontend as separate applications.
2. Use real REST APIs between Flutter and Node.js.
3. Use MySQL for persistent application data.
4. Implement real authentication and authorization.
5. Implement server-side role and permission enforcement.
6. Implement resource ownership checks on every private resource.

7. Use database transactions for financial and inventory-critical operations.
8. Use migrations for database schema management.
9. Use validation on every appropriate API input.
10. Implement centralized API error handling.
11. Implement structured API logging.
12. Document APIs using OpenAPI/Swagger.
13. Implement responsive Flutter interfaces.
14. Support Android and iOS.
15. Support RTL and LTR.
16. Support internationalization.
17. Support light and dark themes.
18. Keep business logic out of Flutter UI widgets.
19. Keep business rules on the backend.
20. Never trust IDs, prices, totals, permissions, roles, or inventory values supplied by the client.
21. Never expose passwords, password hashes, JWT secrets, refresh tokens, payment credentials, or sensitive internal data.
22. Write automated tests for critical business logic.
23. Ensure the project can actually be run locally from a clean environment.

If a requirement cannot yet be connected to an external provider, create a clean provider interface and working mock/test adapter rather than pretending that the integration is real.

2. EXACTLY THREE ACCOUNT TYPES

The platform has exactly three account types:

CUSTOMER

MERCHANT

ADMIN

Do not create additional user-facing account types.

Their permissions must remain strictly separated.

CUSTOMER

A customer buys products.

MERCHANT

A merchant sells products and manages their own store.

ADMIN

An admin controls the marketplace.

Authorization MUST be enforced by the backend.

3. TECHNOLOGY STACK

Flutter:

- Flutter
- Dart
- Clean Architecture
- Feature-based architecture
- Riverpod
- GoRouter
- Dio
- Secure token storage
- Responsive layouts
- Android and iOS
- RTL/LTR
- Internationalization
- Light/dark themes

Backend:

- Node.js
- TypeScript
- Express.js
- REST API
- Clean Architecture
- Modular architecture
- JWT
- Access tokens
- Refresh tokens
- RBAC
- Service layer
- Repository/data-access layer
- Validation middleware
- Authentication middleware
- Authorization middleware
- Ownership checks
- Centralized error handling
- Structured API logging
- OpenAPI/Swagger

Recommended Flutter structure:

```
lib/  
  ■■■ core/  
  ■ ■■■ constants/  
  ■ ■■■ errors/  
  ■ ■■■ network/  
  ■ ■■■ router/
```

```

■ ■■■ storage/
■ ■■■ theme/
■ ■■■ localization/
■ ■■■ utils/
■
■■■■ features/
■ ■■■ auth/
■ ■■■ customer/
■ ■■■ merchant/
■ ■■■ admin/
■ ■■■ products/
■ ■■■ categories/
■ ■■■ search/
■ ■■■ cart/
■ ■■■ wishlist/
■ ■■■ compare/
■ ■■■ checkout/
■ ■■■ orders/
■ ■■■ payments/
■ ■■■ shipping/
■ ■■■ returns/
■ ■■■ reviews/
■ ■■■ promotions/
■ ■■■ messaging/
■ ■■■ notifications/
■ ■■■ support/
■
■■■■ main.dart

```

Recommended backend structure:

```

src/
■■■ config/
■■■ core/
■ ■■■ auth/
■ ■■■ database/
■ ■■■ errors/
■ ■■■ middleware/
■ ■■■ logging/
■ ■■■ utils/
■
■■■■ modules/
■ ■■■ auth/
■ ■■■ users/
■ ■■■ customers/
■ ■■■ merchants/
■ ■■■ products/
■ ■■■ categories/
■ ■■■ brands/
■ ■■■ attributes/
■ ■■■ search/
■ ■■■ cart/
■ ■■■ wishlist/
■ ■■■ compare/
■ ■■■ checkout/
■ ■■■ orders/
■ ■■■ payments/
■ ■■■ shipments/
■ ■■■ returns/
■ ■■■ refunds/
■ ■■■ reviews/
■ ■■■ coupons/
■ ■■■ promotions/
■ ■■■ messaging/
■ ■■■ notifications/
■ ■■■ support/
■ ■■■ analytics/
■ ■■■ payouts/
■ ■■■ admin/
■
■■■■ app.ts

```

Each module should consistently contain:

- controller

- service
- repository
- routes
- dto
- validators
- types
- tests

4. DATABASE

Use MySQL.

The database must be:

- Relational
- Normalized
- Indexed
- Constraint-driven
- Transaction-safe
- Migration-based
- Foreign-key protected
- Designed for concurrency

Use:

- Primary keys
- Foreign keys
- Unique constraints
- Appropriate indexes
- Check constraints where supported
- Created timestamps
- Updated timestamps
- Soft deletion where appropriate
- Transactions
- Row locking where required

Prepare Redis-ready abstractions for:

- Caching
- Rate limiting
- Sessions
- Frequently accessed data
- Background jobs

Use object/external storage for large media.

5. AUTHENTICATION

Implement:

- Customer registration
- Merchant registration
- Login
- Logout
- Access tokens
- Refresh tokens
- Token rotation where appropriate
- Session management
- Session revocation
- Forgot password
- Reset password
- Change password
- Email verification
- Phone verification architecture
- Account deletion

Customer registration must NEVER create an admin.

Admin accounts must be provisioned through a secure administrative mechanism, not public registration.

Use secure password hashing.

6. CUSTOMER FEATURES

Customer profile:

- Full name
- Email
- Phone
- Password
- Profile image
- Country
- City
- Account status
- Verification status

Home page:

- Search
- Search suggestions
- Categories

- Subcategories
- Promotional banners
- Flash sales
- Deals of the day
- Featured products
- Trending products
- Best-selling products
- New arrivals
- Recommended products
- Featured merchants
- Popular brands
- Recently viewed products
- Personalized recommendations
- Coupon promotions
- Seasonal campaigns

Home-page content must be controllable by administrators.

7. DYNAMIC CATALOG

Categories MUST NOT be hard-coded.

Admins must be able to dynamically create unlimited:

- Categories
- Subcategories
- Brands
- Attributes
- Attribute values
- Filters

Initial categories may include:

- Phones
- Tablets
- Laptops
- Computers
- PC components
- Monitors
- Gaming
- Cameras
- Headphones
- Smartwatches
- TVs

- Electronics
- Accessories
- Home appliances
- Furniture
- Clothing
- Shoes
- Watches
- Jewelry
- Beauty
- Personal care
- Sports
- Automotive
- Tools
- Books
- Toys
- Baby products
- Pet products
- Grocery
- Kitchen
- Office supplies
- Industrial products
- Other merchandise

These are seed categories only. The system must remain dynamic.

8. PRODUCT SYSTEM

Products support:

- Name
- Description
- Images
- Videos
- Brand
- SKU
- Barcode
- Price
- Original price
- Discount
- Discount percentage
- Stock status
- Variants

- Specifications
- Dynamic attributes
- Warranty
- Return policy
- Shipping information
- Estimated delivery
- Merchant
- Merchant rating
- Product rating
- Reviews
- Questions and answers
- Related products
- Similar products
- Frequently bought together
- Recently viewed products

Customers can:

- Add to cart
- Buy now
- Wishlist
- Compare
- Share
- Ask questions
- Review
- Report
- Follow merchant

9. DYNAMIC ATTRIBUTES

Attributes must be category-dependent.

PHONE:

Brand, Model, RAM, Storage, Color, Screen Size, Battery, Camera, Processor

LAPTOP:

Brand, CPU, RAM, Storage, GPU, Screen Size, Resolution, Operating System

CLOTHING:

Brand, Size, Color, Material, Style, Gender

Administrators must be able to add new attributes without changing source code.

10. PRODUCT VARIANTS

Support:

- Color
- Size
- Storage
- RAM
- Capacity
- Model
- Configuration

Every variant can have:

- SKU
- Price
- Discount
- Stock
- Barcode
- Images
- Weight
- Dimensions
- Variant-specific attributes

11. SEARCH

Implement scalable search supporting:

- Keyword search
- Search suggestions
- Auto-complete
- Search history
- Popular searches
- Product name
- SKU
- Brand
- Category
- Attributes
- Typo-tolerant architecture
- Filters
- Sorting
- Price range
- Rating

- Brand
- Merchant
- Availability
- Discount
- Product condition
- Delivery options

Results support:

- Grid
- List
- Pagination
- Infinite scrolling
- Sorting
- Filtering
- Skeleton loading
- Empty states

Keep the architecture extensible for a dedicated search engine later.

12. CART

Implement a true multi-merchant cart.

Support:

- Add item
- Remove item
- Quantity changes
- Variant selection
- Stock validation
- Seller grouping
- Shipping calculation
- Coupons
- Discounts
- Taxes
- Delivery fees
- Save for later
- Out-of-stock handling

Never allow a customer to purchase more inventory than is available.

13. CHECKOUT

Flow:

Cart



Address



Shipping



Payment



Order Review



Confirmation

Support:

- Guest checkout where appropriate
- Saved addresses
- New addresses
- Delivery instructions
- Shipping methods
- Coupons
- Discounts
- Taxes
- Delivery fees
- Seller-specific shipping
- Order summary

The server must recalculate final totals. Never trust client-calculated totals.

14. PAYMENTS

Use a provider-independent payment architecture.

Support adapters/interfaces for:

- Credit/debit cards
- Digital wallets
- Cash on delivery
- Bank transfer
- Local payment providers
- Online gateways

Never store raw card data.

States:

PENDING

PROCESSING

PAID

FAILED

CANCELLED

REFUNDED

PARTIALLY_REFUNDED

Implement:

- Server-side verification
- Transaction records
- Idempotency
- Webhooks
- Webhook signature verification
- Refunds

15. ORDERS

Customers can view:

- All
- Pending
- Confirmed
- Processing
- Shipped
- Delivered
- Cancelled
- Returned
- Refunded

Orders include:

- Order number
- Products
- Merchant
- Quantity
- Price
- Discount
- Shipping
- Tax
- Total
- Payment status
- Delivery status
- Tracking number

- Timeline
- Invoice
- Return information
- Refund information

16. ORDER DATA INTEGRITY

Historical orders MUST remain accurate after product changes/deletion.

Store immutable historical information such as:

- Product name
- SKU
- Variant
- Merchant name
- Price
- Quantity
- Discount
- Tax
- Product snapshot

Never reconstruct historical orders from mutable current product data.

17. INVENTORY SAFETY

Inventory operations must be transactional.

Prevent:

- Negative inventory
- Overselling
- Duplicate deductions
- Race conditions
- Incorrect reservations

Use:

- Database transactions
- Row-level locking where appropriate
- Atomic updates
- Reservation logic
- Idempotency

18. WISHLISTS

Support:

- Multiple wishlists
- Add/remove
- Price-drop alerts
- Back-in-stock alerts
- Share wishlist
- Move to cart

19. RETURNS AND REFUNDS

Customers can request:

- Cancellation
- Return
- Refund

Return request contains:

- Order
- Product
- Quantity
- Reason
- Description
- Optional images
- Status

States:

REQUESTED

APPROVED

REJECTED

PICKUP

RECEIVED

REFUND_PENDING

REFUNDED

CLOSED

Support full/partial refunds, deadlines, merchant approval, admin intervention, and refund tracking.

20. REVIEWS

Customers can:

- Rate products
- Write reviews
- Upload images

- Upload videos where supported
- Edit reviews
- Delete reviews
- Report reviews

Support:

- 1–5 stars
- Rating breakdown
- Verified purchase
- Helpful votes
- Moderation
- Merchant responses

Merchants must not alter customer ratings.

21. MERCHANT REGISTRATION

Support:

- Full name
- Business/store name
- Email
- Phone
- Password
- Business type
- Business address
- Country
- City
- Business description
- Store logo
- Store banner
- Verification documents
- Payment/payout information

States:

PENDING

UNDER_REVIEW

APPROVED

REJECTED

SUSPENDED

Selling eligibility is enforced server-side.

22. MERCHANT STORE

Every merchant gets a public storefront containing:

- Store name
- Logo
- Banner
- Description
- Rating
- Products
- Categories
- Promotions
- Reviews
- Shipping policy
- Return policy
- Store information

Customers can visit, follow, view products, contact, and rate the store.

23. MERCHANT DASHBOARD

Display:

- Today's sales
- Total sales
- Revenue
- Orders
- Products
- Customers
- Pending orders
- Low stock
- Out of stock
- Returns
- Refunds
- Notifications
- Sales charts
- Top products

All dashboard metrics come from backend APIs.

24. MERCHANT PRODUCT MANAGEMENT

Merchants can:

- Create

- Edit
- Delete
- Save drafts
- Submit for approval
- Manage images/videos
- Categories
- Attributes
- Variants
- SKUs
- Prices
- Discounts
- Inventory
- Warranty
- Shipping

A merchant MUST NOT manage another merchant's products.

25. MERCHANT INVENTORY

Track:

- Available stock
- Reserved stock
- Sold stock
- Low-stock threshold
- Out-of-stock
- Inventory history
- Stock adjustments
- Bulk updates

Generate low-stock notifications.

26. MERCHANT ORDERS

Merchants can only access order items belonging to their products.

Workflow:

NEW



CONFIRMED



PROCESSING



PACKED

↓
SHIPPED
↓
DELIVERED

Also support cancellation, return, refund, dispute.

27. MERCHANT PROMOTIONS

Support:

- Product discounts
- Store discounts
- Coupons
- Flash sales
- Buy-more-save-more
- Free shipping

Admins define marketplace rules.

28. MERCHANT ANALYTICS

Display:

- Revenue
- Orders
- Products sold
- Top products
- Daily/weekly/monthly/yearly sales
- Average order value
- Refunds
- Cancellations
- Inventory performance

29. MERCHANT PAYOUTS

Track:

Gross Sales
↓
Platform Commission
↓
Refunds / Adjustments
↓
Merchant Earnings
↓
Payout

Track:

- Gross sales
- Commission
- Discounts
- Refunds
- Net earnings
- Pending payout
- Available payout
- Completed payouts
- Failed payouts

All financial calculations are server-side.

30. ADMIN SYSTEM

Admin functionality is completely separated from customer and merchant functionality.

Create a professional admin interface.

31. ADMIN DASHBOARD

Display:

- Total revenue
- Total orders
- Total customers
- Total merchants
- Active merchants
- Pending merchants
- Total products
- Pending products
- Sales
- Refunds
- Returns
- Platform commissions
- Merchant payouts
- User growth
- Merchant growth
- Product growth
- Sales charts

32. ADMIN CUSTOMER MANAGEMENT

Admins can:

- View
- Search
- Filter
- View details
- Activate
- Suspend
- Manage account status
- View order history
- View account activity

Do not expose unnecessary sensitive data.

33. ADMIN MERCHANT MANAGEMENT

Admins can:

- Approve
- Reject
- Suspend
- Reactivate
- Verify documents
- Review information
- View products
- View orders
- View revenue
- View ratings
- Manage permissions

34. ADMIN PRODUCT MANAGEMENT

Admins can:

- Approve
- Reject
- Remove
- Feature
- Control visibility
- Manage prohibited products
- Edit platform-controlled information
- Review reports

35. ADMIN CATALOG MANAGEMENT

Admins can dynamically manage:

- Categories
- Subcategories
- Brands
- Attributes
- Attribute values
- Filters
- Category images
- Category banners
- Ordering

No source-code changes for normal catalog administration.

36. ADMIN ORDER MANAGEMENT

Admins can view marketplace-wide:

- Customers
- Merchants
- Products
- Payments
- Shipping
- Delivery
- Returns
- Refunds
- Order history
- Audit history

37. ADMIN FINANCE

Implement:

- Platform revenue
- Merchant earnings
- Platform commissions
- Payments
- Refunds
- Payouts
- Pending payouts
- Completed payouts
- Failed payouts

- Financial reports

Commission rules configurable by:

- Merchant
- Category
- Product
- Campaign

Financial calculations must be deterministic, auditable, and server-side.

38. CONTENT MANAGEMENT

Admins can manage:

- Home banners
- Promotional banners
- Featured products
- Featured merchants
- Featured categories
- Announcements
- FAQ
- Help pages
- Terms
- Privacy policy
- About page

39. COUPONS

Support:

- Percentage discounts
- Fixed discounts
- Product coupons
- Category coupons
- Merchant coupons
- Marketplace coupons
- Minimum order amount
- Maximum discount
- Start/end date
- Usage limit
- Per-customer limit
- First-order coupons
- Free shipping

Validate coupons on the backend and prevent abuse.

40. FLASH SALES

Support:

- Start/end time
- Products
- Sale price
- Original price
- Sale inventory
- Countdown
- Maximum quantity/customer

Prevent overselling transactionally.

41. LOYALTY

Optional:

- Points
- Rewards
- Customer tiers
- Bonus points
- Redemption
- Points history
- Referral rewards

Every point change must create an immutable transaction.

42. REFERRALS

Support:

- Referral codes
- Referral links
- Tracking
- Rewards
- History
- Anti-abuse

43. MESSAGING

Support:

```
CUSTOMER ↔ MERCHANT  
CUSTOMER ↔ SUPPORT  
MERCHANT ↔ SUPPORT  
MERCHANT ↔ ADMIN
```

Include:

- Conversations
- Messages
- Read status
- Unread counts
- Attachments
- Notifications
- History

Design backend so Socket.IO/WebSockets can be added without rewriting the domain.

44. SUPPORT TICKETS

Customers can:

- Create ticket
- Select category
- Describe issue
- Upload attachments
- Track ticket
- Reply

States:

OPEN

IN_PROGRESS

WAITING_FOR_CUSTOMER

RESOLVED

CLOSED

45. NOTIFICATIONS

Centralized notification system supporting:

- Push
- In-app
- Email
- SMS

Notify about:

- Orders

- Payments
- Shipping
- Delivery
- Returns
- Refunds
- Promotions
- Price drops
- Back-in-stock
- Messages
- Merchant approval
- Product approval
- Security events

Support notification preferences.

46. ADDRESS MANAGEMENT

Customers can manage multiple addresses.

Fields:

- Country
- State/province
- City
- District
- Street
- Building
- Apartment
- Postal code
- Phone
- Delivery instructions

Support add/edit/delete/default.

47. LOCATION-READY ARCHITECTURE

Prepare for:

- Delivery zones
- Distance calculation
- Merchant locations
- Customer locations
- Delivery tracking
- Pickup locations

- Maps
- Geolocation

Do not hard-wire to one map provider.

48. SHIPPING

Provider-independent architecture supporting:

- Multiple providers
- Merchant-specific shipping
- Delivery zones
- Shipping methods
- Shipping fees
- Free shipping
- Estimated delivery
- Tracking numbers
- Shipment statuses
- Local delivery
- Pickup

49. SEARCH AND RECOMMENDATIONS

Create services for:

- Popular products
- Trending products
- Related products
- Similar products
- Recently viewed
- Frequently bought together
- Personalized recommendations

Start rule-based and keep extensible for future ML/recommendation services.

50. SECURITY

Implement:

- Password hashing
- JWT
- Refresh tokens
- Secure token storage
- RBAC
- Permission checks

- Ownership checks
- Input validation
- SQL injection protection
- XSS protection
- CSRF protection where applicable
- Rate limiting
- Secure headers
- CORS
- File-upload validation
- Authentication middleware
- Authorization middleware
- Audit logs

Never expose:

- Password hashes
- Private tokens
- JWT secrets
- Payment credentials
- Sensitive internal data
- Internal stack traces

Use environment variables/secrets management.

51. RESOURCE OWNERSHIP

Every private resource must verify ownership server-side.

Customer order access:

`order.customer_id === authenticatedCustomerId`

Merchant product access:

`product.merchant_id === authenticatedMerchantId`

Merchant order access must be limited to their own order items.

Never trust client-supplied:

- `userId`
- `merchantId`
- `customerId`
- `productId`
- `orderId`

52. DATABASE TABLES

At minimum:

users

roles

permissions

user_roles

customers

merchants

merchant_documents

merchant_stores

addresses

categories

brands

attributes

attribute_values

products

product_images

product_videos

product_variants

product_variant_attributes

inventory

inventory_transactions

carts

cart_items

wishlists

wishlist_items

product_comparisons

orders

order_items

order_status_history

payments

payment_transactions

shipments

shipment_tracking

returns

return_items

refunds

coupons

coupon_usage

promotions

flash_sales

reviews

review_media

review_votes

conversations

messages

notifications

support_tickets

support_messages

loyalty_accounts

loyalty_transactions

referrals

commissions

merchant_payouts

payout_transactions

banners

homepage_sections

audit_logs

app_settings

Add additional normalized tables when required.

53. API DESIGN

Use versioned REST APIs:

```
/api/v1/auth  
/api/v1/customers  
/api/v1/merchants  
/api/v1/admin  
/api/v1/products  
/api/v1/categories  
/api/v1/brands  
/api/v1/attributes  
/api/v1/cart  
/api/v1/wishlist  
/api/v1/compare
```

```
/api/v1/checkout
/api/v1/orders
/api/v1/payments
/api/v1/shipments
/api/v1/returns
/api/v1/refunds
/api/v1/reviews
/api/v1/coupons
/api/v1/promotions
/api/v1/messages
/api/v1/notifications
/api/v1/support
/api/v1/analytics
/api/v1/payouts
```

Every API must implement appropriate authentication, authorization, validation, ownership checks, pagination, filtering, sorting, consistent errors, and HTTP status codes.

54. API RESPONSE FORMAT

Success:

```
{
  "success": true,
  "message": "Request successful",
  "data": {},
  "meta": {}
}
```

Error:

```
{
  "success": false,
  "message": "Validation failed",
  "errors": [
    {
      "field": "email",
      "message": "Invalid email address"
    }
  ]
}
```

Never expose stack traces in production.

55. IDEMPOTENCY AND CONCURRENCY

Critical operations must be safe against retries and concurrent requests.

Use idempotency where appropriate for:

- Payments
- Checkout
- Order creation
- Webhooks
- Refunds
- Payouts
- Inventory operations

Protect against duplicate requests, duplicate webhooks, concurrent purchases, and duplicate refunds.

56. FILE AND MEDIA UPLOADS

Validate:

- MIME type
- File extension
- File size
- Image dimensions where appropriate

Do not trust client filenames/MIME values.

Use external/object storage for large files.

Generate safe storage keys.

Keep private documents access-controlled.

57. AUDIT LOGGING

Audit security-sensitive and administrative actions.

Track:

- Actor
- Action
- Resource
- Resource ID
- Timestamp
- Relevant metadata
- IP/device information where appropriate

58. FLUTTER NAVIGATION

Navigation must be role-aware.

Customer, merchant, and admin navigation must be separate.

Admin screens must never become accessible merely because a Flutter route is manually opened.

Handle:

- Authentication state
- Token expiration
- Refresh
- Logout
- Unauthorized responses
- Role guards
- Deep links

59. UI/UX

Create a modern marketplace design system with:

- Responsive layouts
- Reusable components
- Loading states
- Skeleton states
- Empty states
- Error states
- Pull-to-refresh
- Pagination/infinite scroll
- Form validation
- Confirmation dialogs
- Accessible controls
- RTL/LTR
- Dark/light mode
- Localization

60. INTERNATIONALIZATION

Do not hard-code user-facing strings.

Support:

- Multiple languages
- RTL/LTR
- Locale-aware dates
- Locale-aware numbers
- Currency formatting

61. TESTING

Backend tests:

- Authentication
- Authorization
- Ownership
- Product creation/approval
- Cart
- Checkout
- Inventory
- Concurrent purchases
- Coupons

- Payments
- Orders
- Returns
- Refunds
- Merchant isolation
- Admin permissions

Flutter tests:

- Authentication
- Navigation
- State management
- Forms
- Product browsing
- Cart
- Checkout
- Orders
- Role-specific screens
- Error handling

Critical workflows require integration tests.

62. DATABASE TESTING

Test:

Stock = 1

Customer A purchases

Customer B purchases simultaneously

The system must never create two successful purchases for one available unit.

Also test rollback, duplicate requests, and payment webhook retries.

63. DEVELOPMENT ENVIRONMENT

Provide:

- Environment variables example
- MySQL setup
- Redis-ready setup
- Migrations
- Seed scripts
- API startup
- Flutter startup
- Test commands

- Build commands

Include where appropriate:

.env.example

README.md

docker-compose.yml

Never commit real secrets.

64. SEED DATA

Provide development seed data for:

- One admin
- At least one merchant
- At least one customer
- Categories
- Brands
- Attributes
- Products
- Variants
- Inventory
- Example promotions
- Example banners

Seed data is for development/testing only.

65. OBSERVABILITY

Prepare structured logs for:

- Requests
- Authentication events
- Authorization failures
- Payments
- Orders
- Inventory
- Webhooks
- Errors
- Administrative actions

Never log:

- Passwords
- Access tokens
- Refresh tokens

- Card numbers
- CVV
- Credentials

66. ERROR HANDLING

Centralized backend error handling.

Error categories:

- ValidationError
- AuthenticationError
- AuthorizationError
- NotFoundError
- ConflictError
- BusinessRuleError
- PaymentError
- InventoryError
- ExternalServiceError

Never expose stack traces, SQL statements, internal paths, or secrets.

67. PERFORMANCE

Use:

- Database indexes
- Efficient queries
- Pagination
- Cursor pagination where appropriate
- Caching interfaces
- Lazy loading
- Image optimization
- Background jobs
- Connection pooling

Avoid N+1 queries.

68. BACKGROUND JOBS

Prepare jobs for:

- Emails
- Notifications
- SMS
- Price-drop alerts

- Back-in-stock alerts
- Recommendation calculations
- Analytics aggregation
- Image processing
- Expired promotions
- Flash-sale transitions
- Payout processing

Keep job interfaces independent from HTTP.

69. EXTERNAL PROVIDER ARCHITECTURE

Use interfaces/adapters for:

PaymentProvider

ShippingProvider

EmailProvider

SmsProvider

StorageProvider

NotificationProvider

MapsProvider

SearchProvider

Business logic must depend on interfaces, not vendors.

70. IMPLEMENTATION RULES

DO NOT:

- Build only screens
- Use fake API responses
- Hard-code categories into Flutter
- Hard-code product catalogs
- Store business data only locally
- Trust client prices
- Trust client roles
- Trust client merchant IDs
- Trust client inventory
- Allow cross-merchant access
- Allow cross-customer private-data access
- Create admins through public registration
- Store raw card information
- Skip transactions for inventory/financial operations
- Treat hidden buttons as authorization

- Leave critical features as unexplained TODO placeholders

If an external integration is unavailable, implement the abstraction and a clearly labeled development adapter.

71. IMPLEMENTATION STRATEGY

Build incrementally in vertical working phases.

Phase 1 — Foundation:

Repository, Flutter, backend, MySQL, migrations, configuration, core architecture, errors, logging, API conventions.

Phase 2 — Authentication:

Users, roles, permissions, registration, login, refresh tokens, logout, verification, password reset, sessions.

Phase 3 — Catalog:

Categories, brands, attributes, products, variants, media, inventory.

Phase 4 — Customer shopping:

Home, search, product details, wishlist, comparison, cart, addresses, checkout.

Phase 5 — Orders/payments:

Orders, inventory reservations, payment architecture, webhooks, shipments, tracking, returns, refunds.

Phase 6 — Merchant:

Store, dashboard, products, inventory, orders, promotions, analytics, payouts.

Phase 7 — Admin:

Dashboard, customers, merchants, products, catalog, orders, finance, CMS, audit logs.

Phase 8 — Advanced:

Reviews, messaging, support, notifications, loyalty, referrals, flash sales, recommendations.

Phase 9 — Production hardening:

Security, concurrency tests, integration tests, performance, monitoring, backups, deployment.

72. DEFINITION OF DONE

A feature is NOT complete merely because a screen exists.

A feature is complete only when the necessary flow is connected:

Flutter UI



Riverpod state



Repository



Dio/API client



REST endpoint



Authentication



Authorization



Validation



Service/business logic



Repository



MySQL

For every major feature verify:

- UI exists
- API exists
- Database persistence exists
- Validation exists
- Authorization exists
- Ownership exists
- Error handling exists
- Loading states exist
- Empty states exist
- Tests exist
- Documentation exists

73. REQUIRED DELIVERABLES

Produce a complete runnable project containing:

```
/mobile  
/backend  
/database or /migrations  
/docs  
/infrastructure
```

Include:

- Flutter source
- Node.js/TypeScript source
- MySQL migrations
- Seed scripts
- API documentation
- Environment example

- Docker/development configuration where appropriate
- Automated tests
- README
- Architecture documentation

README must explain:

1. Prerequisites
2. Installation
3. Environment variables
4. Database setup
5. Migrations
6. Seed data
7. Backend startup
8. Flutter startup
9. Test execution
10. API documentation
11. Production configuration

74. FINAL EXECUTION RULE

Do not respond with a high-level plan only.

When asked to build the project, actually create the implementation files and connect the components.

Work systematically.

Before implementing a feature, inspect the existing architecture and reuse existing abstractions.

Do not duplicate functionality.

Do not silently remove requirements.

If requirements conflict, identify the conflict explicitly and choose the architecture that preserves data integrity and security.

If something is ambiguous, choose a reasonable production-safe implementation and document the decision.

At every stage, keep the application buildable.

ULTIMATE GOAL

A REAL

PRODUCTION-READY

MULTI-VENDOR MARKETPLACE

Flutter



REST API

↕

Node.js + TypeScript + Express

↕

Services

↕

Repositories

↕

MySQL

with exactly:

CUSTOMER

MERCHANT

ADMIN

and strict server-side separation between their permissions, data, workflows, dashboards, and resources.